

FlashAttention

Same math, less memory · video 2:46

This lesson answers one question: how can attention avoid storing its huge matrix of scores?

FlashAttention reads the scores in tiles and uses an **online softmax** (a running max, sum and output, rescaled whenever a larger score appears). The result is exactly standard attention, but the biggest piece ever stored is one tile. Measured: 11 MiB instead of 1,557 MiB at 4,096 tokens, and 12 times faster.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The matrix, and the obstacle	▶ 0:08-0:43	2	☐
2. Online softmax	▶ 0:43-1:07	2	☐
3. Tiles, and the same math	▶ 1:07-1:38	3	☐
4. Measured, and why it is faster	▶ 1:38-2:24	2	☐

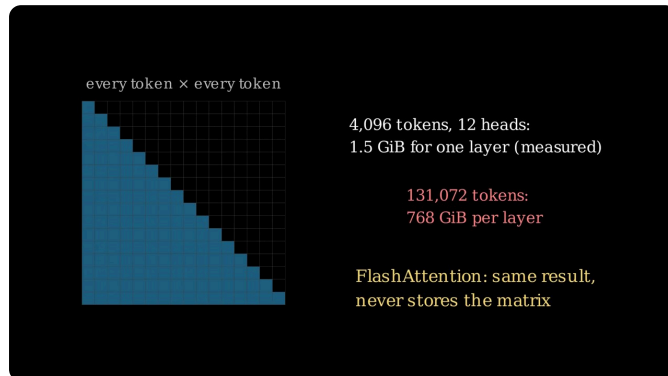
Before you start: Deep Dive episodes 7-8 (the mask; one head, entry by entry). Code: `code/d10_flash_attention` (PyTorch only, no downloads; Linux for the memory measurement).

CONCEPT 1 OF 4

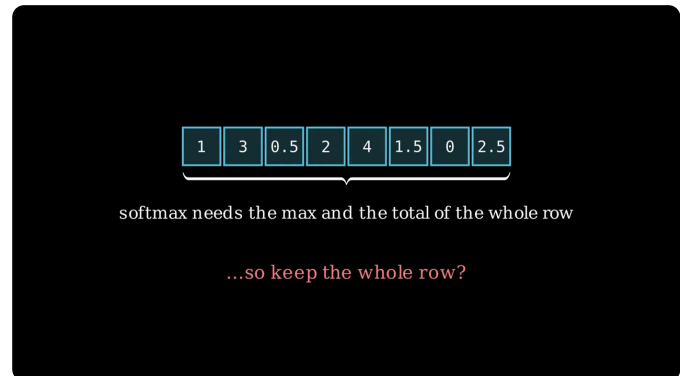
The matrix, and the obstacle

▶ Video 0:08–0:43

Standard attention builds the full matrix of scores, every token against every token. For 4,096 tokens and 12 heads, that is about **1.5 GiB** (measured) for a single layer; at 131,072 tokens it would be **768 GiB**. FlashAttention computes exactly the same result without ever storing that matrix.



▶ 0:29 Standard attention stores every score: 1.5 GiB for 4,096 tokens and 12 heads, in one layer.



▶ 0:42 Softmax needs the largest score and the total of the whole row before any weight is known.

The obstacle is **softmax**: to turn a row of scores into weights you need the row's largest score (for numerical stability) and its total, before any single weight is known. So it seems the whole row must be kept.

KEY IDEA

The score matrix grows with the square of the length; storing it is what makes long contexts run out of memory.

Check 1: The matrix, and the obstacle

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many GiB would the score matrices of one layer take for 8,192 tokens and 12 heads, in float32?

1.2 TRUE OR FALSE

“Doubling the length doubles the memory of the score matrix.”

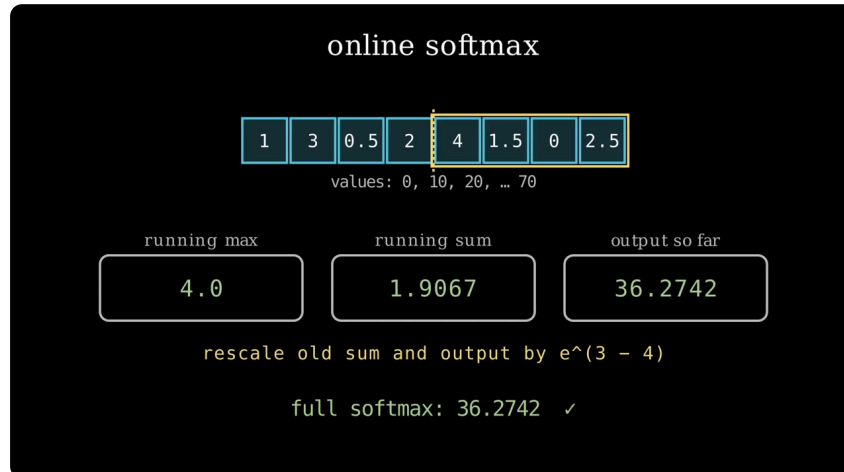
True ☐ False ☐

CONCEPT 2 OF 4

Online softmax

▶ Video 0:43-1:07

Read the row in chunks and keep three running numbers: the **largest score** so far (m), the **sum** so far (l) and the **output** so far. With scores 1, 3, 0.5, 2 | 4, 1.5, 0, 2.5 and values 0, 10, ..., 70: after the first chunk, $m = 3.0$, $l = 1.5853$ and the output is 14.3052. The second chunk brings a 4, so the old sum and output are **rescaled** by $e^{3 - 4}$ before the new terms are added: $m = 4.0$, $l = 1.9067$, output **36.2742**, exactly the full softmax's answer.



▶ 1:06 Eight scores in two chunks: running max 4.0, sum 1.9067, output 36.2742, the same as the full softmax.

KEY IDEA

A softmax-weighted average can be computed in one pass, chunk by chunk: when the max grows, multiply what you have by $e^{\text{old max} - \text{new max}}$.

Check 2: Online softmax

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Scores 1, 2 | 3 with values 0, 10 | 20. After the first chunk (max 2), what is the output so far? And the final output?

2.2 SHORT ANSWER

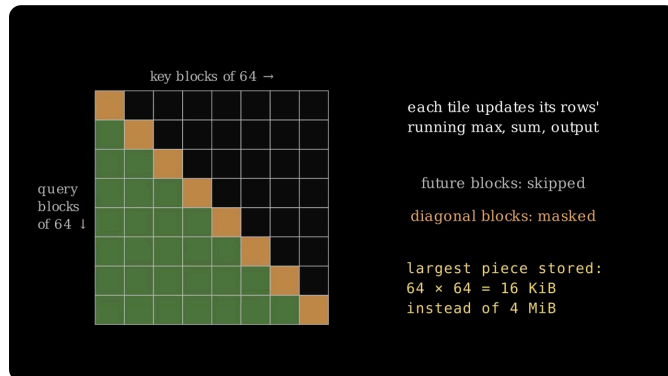
Why subtract the running max at all, instead of just adding up e^{score} ?

CONCEPT 3 OF 4

Tiles, and the same math

▶ Video 1:07-1:38

FlashAttention applies the online softmax to the whole matrix, in **tiles**: a block of 64 queries against a block of 64 keys; compute their scores, update each row's running max, sum and output, move on. Blocks entirely in the future are skipped; only the diagonal blocks need the mask. The biggest piece ever stored is 64×64 numbers, **16 KiB**, instead of $1,024 \times 1,024 = 4 \text{ MiB}$ for one head of 1,024 tokens.



not an approximation

20-line tiled version vs standard:
largest difference 4.8×10^{-7} (rounding)

▶ 1:29 Blocks of 64 queries $\times$ 64 keys; future blocks skipped, diagonal blocks masked; the largest piece stored is 16 KiB.

▶ 1:37 The 20-line tiled version matches standard attention to within 4.8×10^{-7} .

It is not an approximation: the episode's 20-line version matches standard attention to within **4.8×10^{-7}** , float rounding.

KEY IDEA

Tiling plus online softmax gives exact attention with memory that no longer grows with T^2 .

Check 3: Tiles, and the same math

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

With 1,024 tokens and blocks of 64, how many query-key blocks does causal tiled attention compute (out of 256)?

3.2 CALCULATION

How large is one tile of scores with blocks of 128, in float32?

3.3 HANDS-ON CODE

Try it yourself. Run `tiled` from `flash_attention.py` with blocks of 128 and 16. Does the output change?

```
for B in (128, 16):
    b, biggest = tiled(q, k, v, B=B)
    print(B, (a - b).abs().max().item(), biggest)
```

CONCEPT 4 OF 4

Measured, and why it is faster

▶ Video 1:38-2:24

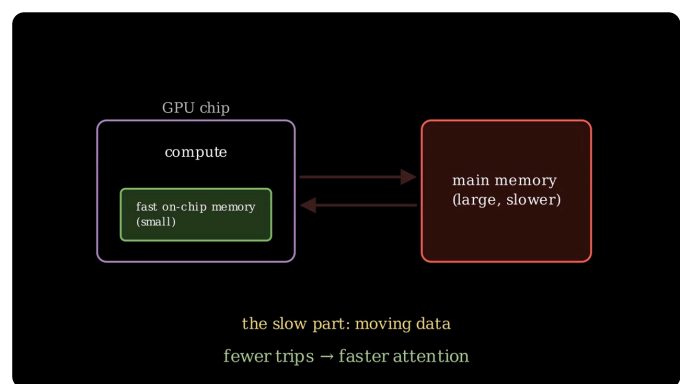
With 12 heads, against PyTorch's fused attention kernel (`scaled_dot_product_attention` , which also works block by block), the extra memory is:

TOKENS	STANDARD	FUSED
1,024	99 MiB	8 MiB
2,048	386 MiB	4 MiB
4,096	1,557 MiB	11 MiB

and at 4,096 tokens it is **12 times faster** ($0.74\text{ s} \rightarrow 0.06\text{ s}$, CPU). The outputs agree to 7×10^{-7} .



▶ 1:54 12 heads: standard 99, 386, 1,557 MiB; fused 8, 4, 11 MiB. At 4,096 tokens, $0.74\text{ s} \rightarrow 0.06\text{ s}$.



▶ 2:11 On a GPU, moving data to and from main memory is the slow part; tiles stay in fast on-chip memory.

Why faster, if the math is the same? On a GPU the slow part is **moving data**, not multiplying it. Small tiles stay in fast on-chip memory, while the big matrix would travel back and forth to the much slower main memory. What does not change: every query is still compared with every earlier key, so the **work** still grows with the square of the length. Cutting that requires skipping pairs (next episode).

KEY IDEA

FlashAttention saves memory and memory traffic, not arithmetic: the work is still quadratic.

Check 4: Measured, and why it is faster

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

Which statement about FlashAttention is true?

- A. It approximates attention to save time
- B. It reduces the work from T^2 to T
- C. It computes exact attention without storing the $T \times T$ matrix
- D. It only works without a causal mask

4.2 CALCULATION

From 2,048 to 4,096 tokens, by what factor did standard attention's measured memory grow?

.....

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The matrix, and the obstacle · rewatch ▶ 0:08-0:43

1.1 3 GiB. $8,192^2 \times 12 \times 4 \text{ bytes} = 3,221,225,472 \text{ bytes} = 3 \text{ GiB}$.

1.2 False. It is $T \times T$: doubling T multiplies it by 4 ($99 \rightarrow 386 \rightarrow 1,557 \text{ MiB}$ in the measurements).

2. Online softmax · rewatch ▶ 0:43-1:07

2.1 7.3106 after the first chunk; 15.7521 at the end. Chunk 1: $(e^{-1} \cdot 0 + 1 \cdot 10) / (e^{-1} + 1) = 7.3106$. Then rescale by $e^2 - 3$ and add $e^0 \cdot 20$: the same as the full softmax, 15.7521.

2.2 To avoid overflow: e^{score} for large scores is too big for floating point. Subtracting the max keeps every exponent at most 0. The standard softmax does the same; the online version just updates the max as it goes.

3. Tiles, and the same math · rewatch ▶ 1:07-1:38

3.1 136. 16 blocks per side; the lower triangle including the diagonal is $16 \times 17 / 2 = 136$.

3.2 64 KiB. $128 \times 128 \times 4 \text{ bytes} = 65,536 \text{ bytes}$.

3.3 No: the difference stays at float rounding (around 10^{-7}) whatever the block size; only the largest tile changes (16,384 or 256 numbers). The block size is a performance choice, not part of the math.

4. Measured, and why it is faster · rewatch ▶ 1:38-2:24

4.1 C. Same result (to rounding), memory per tile only, and the causal mask is handled by skipping and masking blocks.

4.2 About 4 ($386 \rightarrow 1,557 \text{ MiB}$, $\times 4.03$). The score matrix is $T \times T$.